Lesson 5: Enemy Ranks

GameMaker

VIDEO GAME CREATION

This week: one enemy becomes many. Your job is to turn last week's single enemy into a whole row of them, using a loop instead of repeating yourself by hand.

Open your own [YourName]Game project from Lesson 4 - don't start a new one. Everything this week builds on the enemy and collision detection you already have.

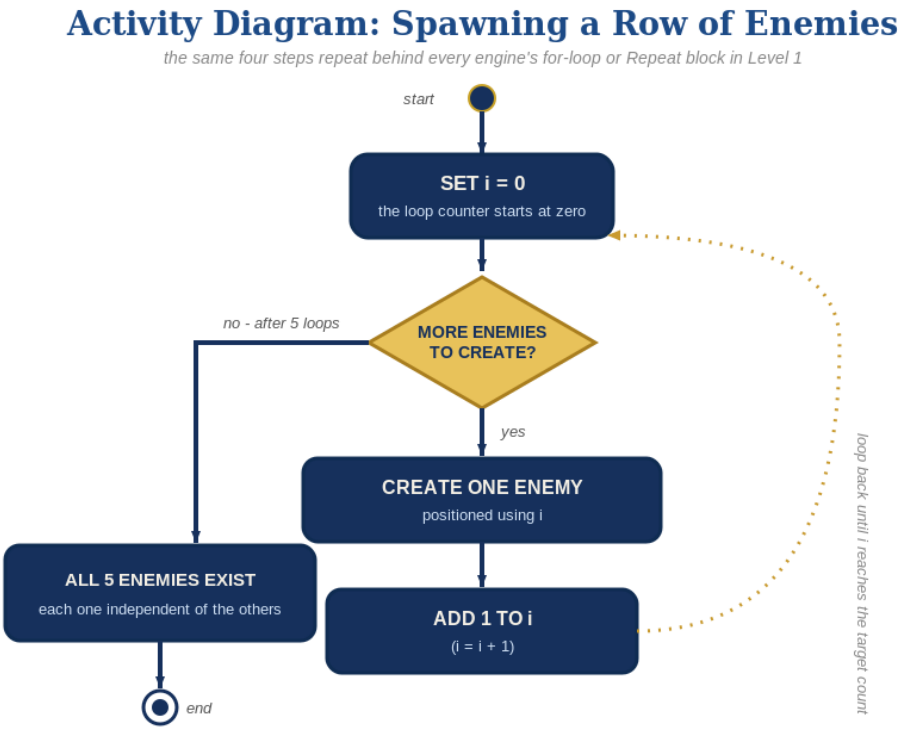
Tip: no new assets this week - you're reusing bullet_yellow.png, sfx_shoot.wav, enemy_invader_red.png, and sfx_hit.wav from your Lesson 3 and 4 Assets folders.

Setup

1. Open your [YourName]Game project from Lesson 4.
2. Remove the single obj_enemy instance you manually placed in your room last week: open the Room Editor, click that instance in the room, and press Delete (or right-click it > Delete). Do NOT delete the obj_enemy object itself from the Asset Browser - obj_spawner still needs that object to exist so it can create new instances of it.
3. Create a new object obj_spawner (no sprite needed), and drag one instance of it into your room.

Level 1 - Minimal (MVP)

Several enemies sit on screen in a row, created by a loop instead of placed or spawned one at a time. Shooting any one of them destroys just that one. This is a complete finish for today - reaching this is enough to move on to next week.



Every engine's Level 1 solution runs this same shape - a counter, a repeat check, one creation step, and an increment - even though the exact blocks or code syntax differ. Tracing a spawn bug means finding which of these four steps broke.

The spawn loop, in general - the same counter, repeat check, creation step, and increment behind Level 1 in every engine, whatever the exact blocks or syntax look like.

4. Add a Create event to obj_spawner with a for loop that calls instance_create_layer() several times at different positions (code below).
5. Press F5 (Run) - you should see a row of enemies, and shooting any one of them should destroy just that one.

```
// obj_spawner's Create event:
for (var enemy_count = 0; enemy_count < 5; enemy_count++) {
    instance_create_layer(80 + enemy_count * 96, 80, "Instances",
obj_enemy);
}
```

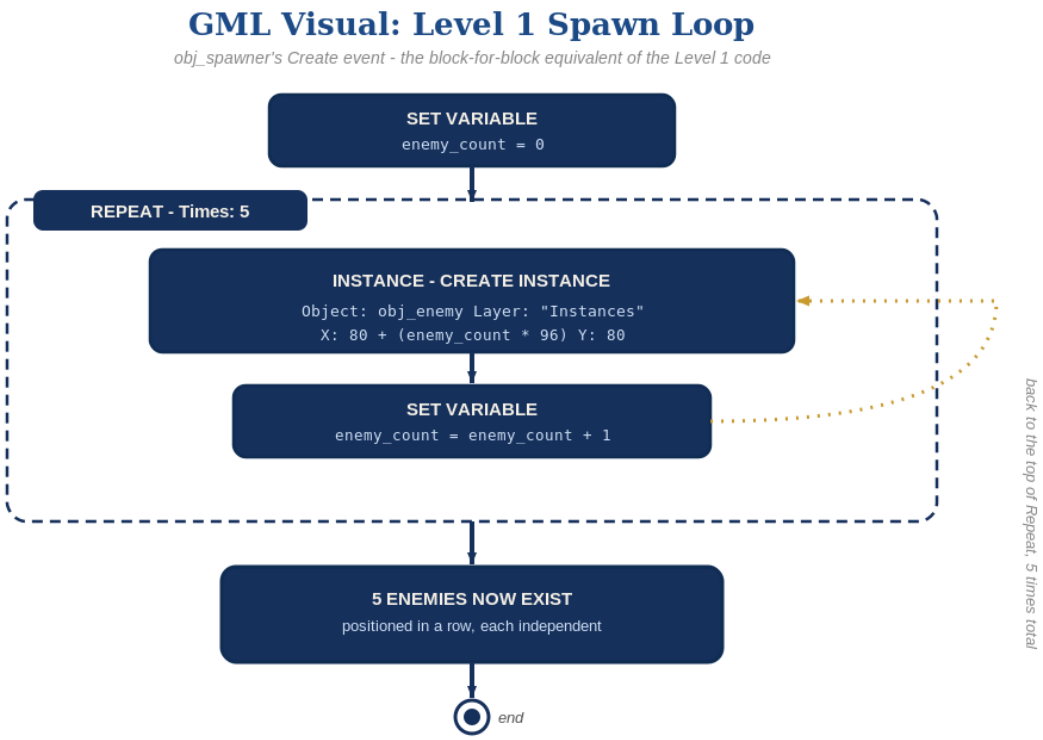
Alternate Path: GML Visual (no typing)

The same for-loop spawn pattern works with drag-and-drop blocks instead of typed code.

6. On obj_spawner's Create event, switch to the GML Visual tab (the toggle at the top-left of the event, next to the code-view icon).
7. From the Variables category, drag a "Set Variable" block into the event. Set its Variable field to enemy_count and its Value field to 0 - this is the loop counter, the block equivalent of var enemy_count = 0 in the code path.
8. From the Control category, drag a "Repeat" block below it. Set its Times field to 5.
9. Inside the Repeat block's body, drag an "Instance - Create Instance" block (from the Instances category). Set its Object field to obj_enemy, its Layer field to "Instances"

(matching the layer name from Lesson 3's setup), its X field to the expression 80 + (enemy_count * 96), and its Y field to 80.

10. Still inside the Repeat block, right after the Create Instance block, drag a second "Set Variable" block: Variable field = enemy_count, Value field = enemy_count + 1 - this increments the counter each pass, the block equivalent of enemy_count++ in the code for loop.
11. Press F5 (Run) and test the same way as the code path: you should see 5 enemies in a row, and shooting any one destroys just that one.



Five blocks, three of them wired inside one Repeat - the exact same shape as the Activity Diagram above, just built with GML Visual's own blocks instead of typed code.

The five blocks above, wired together - the block-for-block equivalent of the Level 1 code.

Level 3 Visual-Track Equivalent

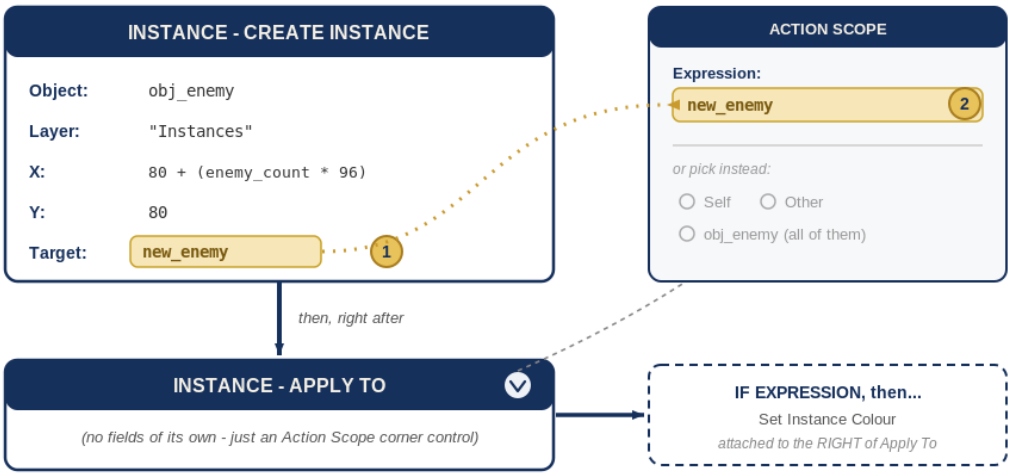
- Sound and Structure need no changes from the code path either way - both already live on obj_enemy and obj_bullet's own events, which behave identically no matter which path spawned the instance.
- Visual (color-switching), on the other hand, DOES need its own block sequence - it happens inside the spawn loop itself, so it's built differently in Visual than in code. Give the "Instance - Create Instance" block a Target field (in its own block, alongside Object/Layer/X/Y) and set it to a new local variable, e.g. new_enemy - this captures a reference to the specific instance that call just created, the block equivalent of var new_enemy = instance_create_layer(...) in the code path (the code path uses this exact same name for it now, too).
- Then drag an "Instance - Apply To" block (Instances category) right after it. Open its Action Scope control (the small dropdown arrow in the block's own top-right corner, the

same control every action block has) - instead of picking one of the listed Self/Other/object options, type new_enemy directly into the Expression field at the top of that pop-up. This re-points "Self" to that one captured instance for everything nested inside Apply To, without needing a wire or socket to carry the reference.

- Attach an "If/Else" block (Control category) to the RIGHT of Apply To, not below it - that's what nests the If/Else inside Apply To's scope rather than just running it after. Set its condition to enemy_count mod 2 = 0.
- Inside the If branch, drag a "Set Instance Colour" block (Instances category) - Colour field: make_color_rgb(200, 25, 55), a deep crimson red; Use Alpha: off. Inside the Else branch, the same block again - Colour: make_color_rgb(230, 100, 130), a lighter dusty rose/pink; Use Alpha: off. Because both live inside Apply To's scope, "Self" already means the one enemy just spawned - no targeting dropdown to fight with.
- This is the purpose-built block after all, not a workaround: Set Instance Colour's own Action Scope dropdown looks like it only offers Self, Other, or a specific object asset (which would recolor every enemy that exists so far, not just the new one) - but nesting it inside an Apply To block whose Expression field names the captured instance variable is what lets "Self" mean that one instance. Apply To is the piece that reaches a dynamically-created instance; once it's in place, every ordinary purpose-built block nested inside it just works normally.

GML Visual Wireframe: Capturing One Spawned Instance

inside the Repeat block, obj_spawner's Create event - Create Instance's Target, handed off to Apply To's Action Scope



1. Target is the only place Create Instance lets you capture a reference to the exact instance it just made.
2. Apply To's Action Scope Expression field is where that captured name goes instead of Self/Other/an object - everything attached to the RIGHT of Apply To (not below it) then runs with "Self" meaning that one instance, so the purpose-built Set Instance Colour block works normally - no generic block or typed workaround needed.

The exact block wiring behind that Target field - Create Instance's Target captures the new instance, and Apply To's Action Scope Expression field is what carries that capture into every block nested to its right.

See the Level 3 Visual-Track Equivalent above for exactly how the color-switching idea maps into GML Visual - it's the one Level 3 idea that isn't identical between Code and Visual, since it lives inside the spawn loop itself.

Level 2 - Complete (Alpha/Beta)

Every enemy in the row can be individually shot and destroyed without affecting the others, and a bullet that misses every enemy still behaves exactly like it did last week.

- Shoot several different enemies in a row, including some misses - each hit destroys exactly the one enemy that was actually touched, and every other enemy is unaffected.
- This already works with no extra code: obj_bullet's Collision event with obj_enemy already reacts per-pair of overlapping instances, so every instance the loop creates behaves independently and correctly, exactly the way Lesson 4 built it.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Sound: this already works with no changes - audio_play_sound() in obj_bullet's Collision event runs independently for every enemy that gets hit.
- Visual: vary each obj_enemy's image_blend based on enemy_count in obj_spawner's loop, using with (instance_create_layer(...)) { image_blend = ... } to set a property on the instance right after creating it, for a striped row effect. make_color_rgb(200, 25, 55) is a deep crimson red and make_color_rgb(230, 100, 130) is a lighter dusty rose/pink - image_blend multiplies your sprite's pixels by this color, so alternating the two tints every other enemy a visibly different shade of red rather than blending into one new color.
- Structure: keep obj_spawner's Create event limited to just creating and positioning enemies - resist the urge to also handle collision-response logic there; that still belongs entirely inside obj_enemy and obj_bullet's own events.

If Something's Wrong

- Only one enemy appears, or they're all stacked in the same spot: confirm the loop variable enemy_count is actually used in the position calculation, not a fixed number.
- "instance_create_layer" errors about a missing layer: confirm the room's instance layer is still actually named "Instances", matching Lesson 3's setup.
- Old and new enemies both appear: confirm the manually-placed Lesson 4 obj_enemy instance was actually deleted from the room.
- "obj_enemy doesn't exist" or obj_spawner's Create event won't compile: this means the obj_enemy object asset itself got deleted from the Asset Browser, not just its placed instance - Object (the reusable template in the Asset Browser) and Instance (one placed copy of it in the room) are different things in GameMaker; you can freely delete an instance from a room without touching the object it was made from, but instance_create_layer() needs the object to still exist. Recreate obj_enemy from Lesson 4's Key if this happens.

Want More?

Student Resource List > GameMaker section, or gamemaker.io/en/tutorials